

Parallel LINQ (PLINQ) in C# with Examples:

In this article, I will discuss **Parallel LINQ (PLINQ) in C#** with Examples. Please read our previous article discussing [Interlocked vs. Lock in C#](#) with Examples.

1. [What is Parallel LINQ \(PLINQ\) in C#?](#)
2. [Example to Understand Parallel LINQ in C#](#)
3. [How to Maintain the Original Order in Parallel LINQ?](#)
4. [Difference Between OrderBy and AsOrdered Method in C#](#)
5. [Maximum Degree of Parallelism and Cancellation Token in Parallel LINQ](#)
6. [Doing Aggregates in Parallel LINQ](#)
7. [Is Really Parallel LINQ Improving the Performance of an Application?](#)
8. [Key Characteristics of Parallel LINQ](#)
9. [When to Use Parallel LINQ in C#?](#)
10. [When Not to Use Parallel LINQ in C#?](#)

What is Parallel LINQ (PLINQ) in C#?

If we have a collection, and if we want to use parallelism to process it, we have the option of using Parallel LINQ or PLINQ. Parallel LINQ (PLINQ) is basically the same as we have in LINQ. But with parallel functionality, we can define the maximum degree of parallelism and use a cancellation token to cancel the operation. To be able to process a sequence with PLINQ, we need to use the `AsParallel` method.

Parallel LINQ (PLINQ) is a parallel implementation of LINQ to Objects that combines the simplicity and readability of LINQ syntax with the power of parallel programming. It is designed to make optimal use of all the cores on a modern processor to improve the performance of LINQ operations on large data sets or CPU-bound operations.

PLINQ achieves parallelism by partitioning the source collection into segments and then processing these segments concurrently across multiple threads. To use PLINQ, you typically need to add a call to the `AsParallel` method to your LINQ query, which instructs the system to process the query in parallel if it's deemed beneficial.

Example to Understand Parallel LINQ in C#:

Let us understand Parallel LINQ in C# with an example. In the below example, we are creating a collection of integer numbers from 1 to 20 using **Enumerable.Range** method. Then, using the LINQ `Where` Extension method, we filter the list of even numbers from the numbers collection. In the example below, we are not using Parallel LINQ; we are simply using LINQ.

```
using System;
using System.Linq;
namespace ParallelLINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating a Collection of integer numbers
```

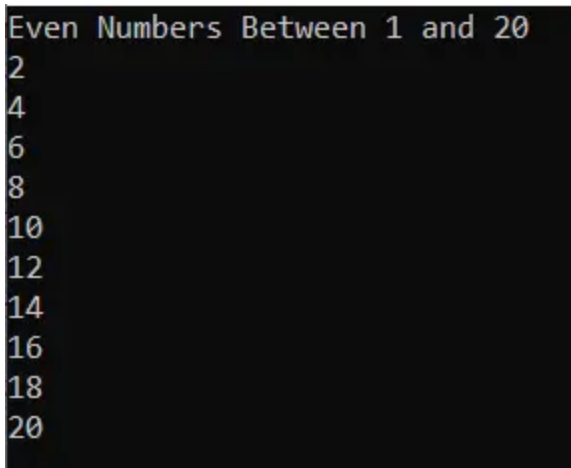
```

var numbers = Enumerable.Range(1, 20);
//Fetching the List of Even Numbers using LINQ
var evenNumbers = numbers.Where(x => x % 2 == 0).ToList();
Console.WriteLine("Even Numbers Between 1 and 20");
foreach (var number in evenNumbers)
{
    Console.WriteLine(number);
}

Console.ReadKey();
}
}
}

```

Output:



```

Even Numbers Between 1 and 20
2
4
6
8
10
12
14
16
18
20

```

Once you run the code, you will get the above output. The following is the code that filters the even numbers using LINQ.

```
var evenNumbers = numbers.Where(x => x % 2 == 0).ToList();
```

Let us see how to use Parallel LINQ in C# with the same example. As discussed earlier, we need to use the `AsParallel` method to process the sequence in parallel. For a better understanding, please look at the code below, which shows both LINQ and PLINQ syntaxes to get even numbers from the numbers collection. As you can see, with Parallel LINQ, after the collection or sequence and before the `Where` extension method, we are calling the `AsParallel` Extension method.

```
//Creating a Collection of integer numbers
```

```
var numbers = Enumerable.Range(1, 20);
```

```
//Fetching the List of Even Numbers using LINQ
```

```
var evenNumbers = numbers.Where(x => x % 2 == 0).ToList();
```

← LINQ

```
//Fetching the List of Even Numbers using PLINQ
```

```
//PLINQ means we need to use AsParallel()
```

```
var evenNumbers = numbers.AsParallel().Where(x => x % 2 == 0).ToList();
```

↑
PLINQ

↑
Using AsParallel method to make it
Parallel LINQ

So, this is as simple as it is. The following code uses parallelism. The evaluations (i.e., `x => x % 2 == 0`) will be done in parallel, i.e., the Where Extension method will be executed parallelly.

```
var evenNumbers = numbers.AsParallel().Where(x => x % 2 == 0).ToList();
```

↑
These evaluations are going to be
done in Parallel

Now, let us iterate over the evenNumbers collection and see the output. The following is the complete code example. The following example code is self-explained, so please go through the comment lines for a better understanding.

```
using System;
```

```
using System.Linq;
```

```
namespace ParallelLINQDemo
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            //Creating a Collection of integer numbers
```

```
            var numbers = Enumerable.Range(1, 20);
```

```
            //Fetching the List of Even Numbers using LINQ
```

```
            //var evenNumbers = numbers.Where(x => x % 2 == 0).ToList();
```

```
            //Fetching the List of Even Numbers using PLINQ
```

```
            //PLINQ means we need to use AsParallel()
```

```
            var evenNumbers = numbers.AsParallel().Where(x => x % 2 == 0).ToLi
```

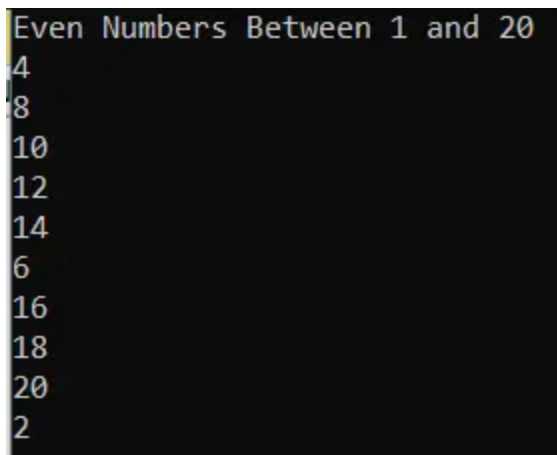
```

        Console.WriteLine("Even Numbers Between 1 and 20");
        foreach (var number in evenNumbers)
        {
            Console.WriteLine(number);
        }

        Console.ReadKey();
    }
}

```

Output:



```

Even Numbers Between 1 and 20
4
8
10
12
14
6
16
18
20
2

```

You can observe the order of the numbers. They are in random order. This is because we have already seen in the past that when we use parallelism, we typically cannot control the order of the operations, i.e., we cannot simply predict the order of the output. Now, if you run the code multiple times, you might get a different order of the numbers each time.

How to Maintain the Original Order in Parallel LINQ?

Suppose you want the output to be in order. In that case, you need to use the `AsOrdered()` method after the `AsParallel()` method, which means that after doing the operations in parallel, it will maintain the original order of the elements as they appeared in the sequence. For a better understanding, please look at the following image, which shows how to use the `AsOrdered` method.

```

var evenNumbers = numbers
    .AsParallel() //Parallel Processing
    .AsOrdered() //Original Order of the numbers
    .Where(x => x % 2 == 0)
    .ToList();

```

The order will be the original order in which the elements are stored in the numbers collections. The following is the complete code. The following example code is self-explained, so please go through the comment lines for a better understanding.

```

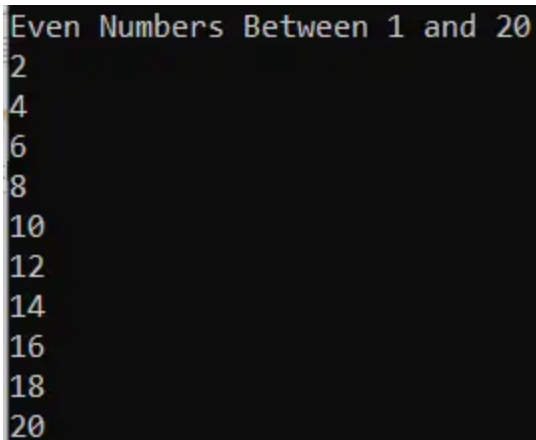
using System;
using System.Collections.Generic;
using System.Linq;
namespace ParalleLLINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating a Collection of integer numbers
            var numbers = Enumerable.Range(1, 20);

            //Fetching the List of Even Numbers using PLINQ
            //PLINQ means we need to use AsParallel()
            var evenNumbers = numbers
                .AsParallel() //Parallel Processing
                .AsOrdered() //Original Order of the numbers
                .Where(x => x % 2 == 0)
                .ToList();
            Console.WriteLine("Even Numbers Between 1 and 20");
            foreach (var number in evenNumbers)
            {
                Console.WriteLine(number);
            }

            Console.ReadKey();
        }
    }
}

```

Output:



```

Even Numbers Between 1 and 20
2
4
6
8
10
12
14
16
18
20

```

Now, you can see the numbers are in the original order. Now, it doesn't matter how many times you run the code. It will always bring up the current order of the elements, which is great in case you need it.

Difference Between OrderBy and AsOrdered Method in C#:

You might be interested in using the LINQ OrderBy method after the Where Extension method. But remember, the OrderBy method is used to sort the data, not to maintain the original order of the data. On the other hand, the AsOrdered method is used to maintain the original order of the elements. For a better understanding, please have a look at the following example. The following example code is self-explained, so please go through the comment lines.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace ParallelLINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating a Collection of integer numbers
            List<int> numbers = new List<int>()
            {
                1, 2, 6, 7, 5, 4, 10, 12, 13, 20, 18, 9, 11, 15, 14, 3, 8, 16,
            };
            //Using AsOrdered Method
            //Fetching the List of Even Numbers using PLINQ
            var evenNumbers1 = numbers
                .AsParallel() //Parallel Processing
                .AsOrdered() //Original Order of the numbers
                .Where(x => x % 2 == 0)
                .ToList();
            Console.WriteLine("Even Numbers Between 1 and 20 using AsOrdered");
            foreach (var number in evenNumbers1)
            {
                Console.WriteLine(number);
            }

            //Using OrderBy Method
            //Fetching the List of Even Numbers using PLINQ
            var evenNumbers2 = numbers
                .AsParallel() //Parallel Processing
                .Where(x => x % 2 == 0)
                .OrderBy(x => x) //Sort the Elements in Ascending Order
                .ToList();
            Console.WriteLine("\nEven Numbers Between 1 and 20 using OrderBy");
            foreach (var number in evenNumbers2)
            {
                Console.WriteLine(number);
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
Even Numbers Between 1 and 20 using AsOrdered
```

```
2  
6  
4  
10  
12  
20  
18  
14  
8  
16
```

```
Even Numbers Between 1 and 20 using OrderBy
```

```
2  
4  
6  
8  
10  
12  
14  
16  
18  
20
```

As you can see in the above output, with the `AsOrdered` method, we are getting the data in its original order of the sequence. With the `OrderBy` method, we are getting the data in ascending order.

Maximum Degree of Parallelism and Cancellation Token in Parallel LINQ:

In [Parallel Programming](#), we have discussed the maximum degree of parallelism and cancellation token. We can have the same functionality here with Parallel LINQ while processing a collection. For example, we can define the maximum degree of parallelism. We can also define and pass a cancellation token that will cancel the execution of the Parallel LINQ operation. For a better understanding, please have a look at the below image. Here, you can see we are creating a `CancellationTokenSource` instance and setting the token to cancel the operation after 200 `Milliseconds`. And then passing the Cancellation Token as a value to the `WithCancellation` method. Further, we are passing a value of 2 to the `WithDegreeOfParallelism` method, which means a maximum of two threads will process the operations.

```

//Creating an instance of CancellationTokenSource
var CTS = new CancellationTokenSource();

//Setting the time when the token is going to cancel the Parallel Operation
CTS.CancelAfter(TimeSpan.FromMilliseconds(200));

//Creating a Collection of integer numbers
var numbers = Enumerable.Range(1, 20);

//Fetching the List of Even Numbers using PLINQ
var evenNumbers = numbers
    .AsParallel() //Parallel Processing
    .AsOrdered() //Original Order of the numbers
    .WithDegreeOfParallelism(2) //Maximum of two threads can process the data
    .WithCancellation(CTS.Token) //Cancel the operation after 200 Milliseconds
    .Where(x => x % 2 == 0) //This logic will execute in parallel
    .ToList();

```

So, with Parallel LINQ in C#, we can achieve the same functionality as parallel programming. The complete example code is given below. The following example code is self-explained, so please go through the comment lines for a better understanding.

```

using System;
using System.Linq;
using System.Threading;
namespace ParallelLINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating an instance of CancellationTokenSource
            var CTS = new CancellationTokenSource();
            //Setting the time when the token is going to cancel the Parallel
            CTS.CancelAfter(TimeSpan.FromMilliseconds(200));
            //Creating a Collection of integer numbers
            var numbers = Enumerable.Range(1, 20);

            //Fetching the List of Even Numbers using PLINQ
            var evenNumbers = numbers
                .AsParallel() //Parallel Processing
                .AsOrdered() //Original Order of the numbers
                .WithDegreeOfParallelism(2) //Maximum of two threads can process
                .WithCancellation(CTS.Token) //Cancel the operation after 200
            Milliseconds
                .Where(x => x % 2 == 0) //This logic will execute in parallel
                .ToList();
            Console.WriteLine("Even Numbers Between 1 and 20");
            foreach (var number in evenNumbers)

```

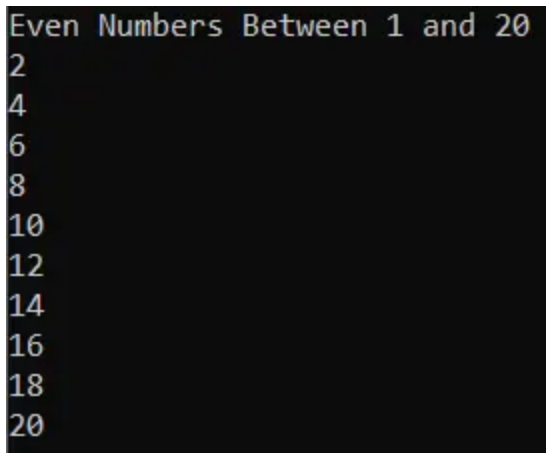
```

        {
            Console.WriteLine(number);
        }

        Console.ReadKey();
    }
}

```

Output:



```

Even Numbers Between 1 and 20
2
4
6
8
10
12
14
16
18
20

```

Doing Aggregates in Parallel LINQ

Let us see how we can aggregate the elements of a sequence using Parallel LINQ in C#. For example, we can calculate the average of the elements of a collection, we can add the elements of a collection, etc. Let us see an example of calculating a collection's Sum, Max, Min, and Average using Parallel LINQ in C#. The following example code is self-explained, so please go through the comment lines for a better understanding.

```

using System;
using System.Linq;
namespace ParallelLINQDemo
{
    class Program
    {
        static void Main()
        {
            var numbers = Enumerable.Range(1, 10000);
            //Sum, Min, Max and Average LINQ extension methods
            Console.WriteLine("Sum, Min, Max and Average with LINQ");

            var Sum = numbers.AsParallel().Sum();
            var Min = numbers.AsParallel().Min();
            var Max = numbers.AsParallel().Max();
            var Average = numbers.AsParallel().Average();
            Console.WriteLine($"Sum:{Sum}\nMin: {Min}\nMax: {Max}\nAverage:{Average}");

            Console.ReadKey();
        }
    }
}

```

```

    }
}
}

```

Output:

```

Sum, Min, Max and Average with LINQ
Sum:50005000
Min: 1
Max: 10000
Average:5000.5

```

Is Really Parallel LINQ Improving the Performance of an Application?

Let us see an example using both LINQ and Parallel LINQ to do the same task and then see the performance benchmark. Please have a look at the below example. In the example below, we compare the performance of LINQ and PLINQ Min, Max, and Average methods. The Min, Max, and Average methods are going to return a single scalar value or, you can say, aggregate value.

```

using System;
using System.Diagnostics;
using System.Linq;
namespace ParallelLINQDemo
{
    class Program
    {
        static void Main()
        {
            var random = new Random();
            int[] values = Enumerable.Range(1, 99999999)
                .Select(x => random.Next(1, 1000))
                .ToArray();
            //Min, Max and Average LINQ extension methods
            Console.WriteLine("Min, Max and Average with LINQ");

            Stopwatch stopwatch = new Stopwatch();
            stopwatch.Start();
            // var linqStart = DateTime.Now;
            var linqMin = values.Min();
            var linqMax = values.Max();
            var linqAverage = values.Average();
            stopwatch.Stop();
            var linqTimeMS = stopwatch.ElapsedMilliseconds;
            DisplayResults(linqMin, linqMax, linqAverage, linqTimeMS);
            //Min, Max and Average PLINQ extension methods
            Console.WriteLine("\nMin, Max and Average with PLINQ");
            stopwatch.Restart();
            var plinqMin = values.AsParallel().Min();
            var plinqMax = values.AsParallel().Max();

```

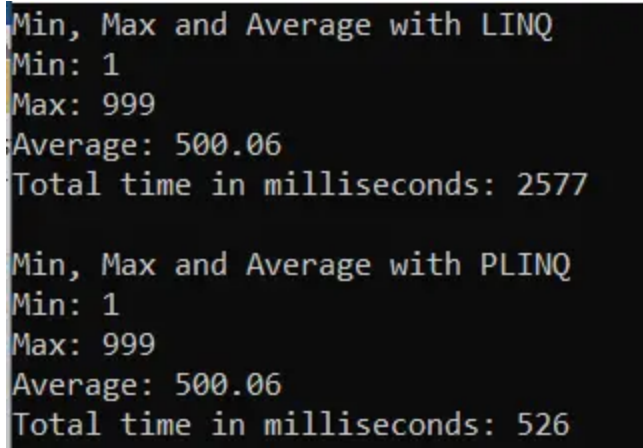
```

        var plinqAverage = values.AsParallel().Average();
        stopwatch.Stop();
        var plinqTimeMS = stopwatch.ElapsedMilliseconds;
        DisplayResults(plinqMin, plinqMax, plinqAverage, plinqTimeMS);

        Console.ReadKey();
    }
    static void DisplayResults(int min, int max, double average, double time)
    {
        Console.WriteLine($"Min: {min}\nMax: {max}\n" + $"Average: {average}\nTotal time in milliseconds: {time}");
    }
}

```

Output:



```

Min, Max and Average with LINQ
Min: 1
Max: 999
Average: 500.06
Total time in milliseconds: 2577

Min, Max and Average with PLINQ
Min: 1
Max: 999
Average: 500.06
Total time in milliseconds: 526

```

Points to Remember When Working with PLINQ:

There are several things to keep in mind when using PLINQ:

- **Ordering:** PLINQ processes elements in parallel and doesn't guarantee the order of the results unless you explicitly ask for it using the `AsOrdered` method. Preserving order can add overhead and potentially reduce performance gains.
- **Side Effects:** Queries should be side-effect-free. Since PLINQ can process multiple elements simultaneously, using functions that have side effects can result in unexpected behavior or race conditions.
- **Exception Handling:** When an exception is thrown in a PLINQ query, it is wrapped in an `AggregateException`. This exception contains all the individual exceptions thrown during the execution of the query.
- **Performance:** Not all queries run faster in parallel. PLINQ has overhead, so for small collections or fast operations, the overhead might outweigh the benefits of parallelism. Measuring performance for sequential and parallel versions of your query is generally a good idea.
- **Forcing Parallelism:** You can force PLINQ to parallelize the query using the `WithExecutionMode` method with `ParallelExecutionMode.ForceParallelism` is not recommended unless you are certain that parallelism will improve performance.
- **Degree of Parallelism:** You can specify the number of processors to use in a PLINQ query by using the `WithDegreeOfParallelism` method. This can be useful for fine-tuning performance, particularly if

you want to leave some cores free for other tasks.

PLINQ is particularly useful when working with large collections or performing complex processing where tasks can be executed in parallel. It is a powerful tool, but it should be used judiciously and tested to ensure that it provides performance benefits in each specific case.

When to Use Parallel LINQ in C#?

Parallel LINQ (PLINQ) in C# should be used when you have computationally intensive queries that can benefit from parallel execution to improve performance. Here are some scenarios where PLINQ is particularly useful:

- **Large Data Sets:** When processing large data collections, PLINQ can significantly reduce the time it takes by dividing the work across multiple threads.
- **CPU-Intensive Operations:** If your LINQ queries involve complex calculations or CPU-bound processing, PLINQ can help distribute the load across multiple cores.
- **Multi-Core Processors:** If the environment where your application is running has multiple CPU cores available, PLINQ can utilize these cores to perform operations in parallel.
- **Long-Running Queries:** For operations that naturally take a long time to execute, parallelizing the work can lead to better application responsiveness, especially in desktop or server environments.

When Not to Use Parallel LINQ in C#?

There are situations where PLINQ may not be appropriate:

- **Small Data Sets:** If the collection is relatively small, the overhead of partitioning the data and managing threads may outweigh the benefits of parallel processing.
- **I/O-Bound Operations:** If your query is I/O-bound, waiting on file system operations or network responses, parallelizing the query won't improve performance and may even degrade it.
- **Ordered Queries:** When the order of results is important, PLINQ can introduce overhead because maintaining order in a parallelized environment is more complex.
- **Non-Thread-Safe Operations:** If your query performs operations that are not thread-safe, such as writing to shared variables without proper synchronization, PLINQ can cause more problems than it solves.
- **Asynchronous Operations:** If you can perform I/O-bound work asynchronously, doing so is often more efficient with `async/await` rather than using PLINQ.

Before using PLINQ, it's important to consider whether the benefits of parallel execution outweigh the overhead. It's also important to test and measure performance because the ideal use case for PLINQ can vary depending on the specifics of the workload and the runtime environment.

In the next article, I will discuss the [Differences Between Multithreading vs. Asynchronous Programming vs. Parallel Programming in C#](#) with Examples. In this article, I try to **Parallel LINQ (PLINQ) in C#** with Examples. I hope you enjoy this Parallel LINQ (PLINQ) in C# with Examples.